

1.7.1 #if, #elif, #else, #endif

These preprocessing directives create conditional compiling parameters that control the compiling of the source code. They must begin on a separate line.

Syntax:

```
#if constant_expression
#else
#endif

or

#if constant_expression
#elif constant_expression
#endif
```

The compiler only compiles the code after the `#if` expression if the *constant_expression* evaluates to a non-zero value (true). If the value is 0 (false), then the compiler skips the lines until the next `#else`, `#elif`, or `#endif`. If there is a matching `#else`, and the *constant_expression* evaluated to 0 (false), then the lines between the `#else` and the `#endif` are compiled. If there is a matching `#elif`, and the preceding `#if` evaluated to false, then the *constant_expression* after that is evaluated and the code between the `#elif` and the `#endif` is compiled only if this expression evaluates to a non-zero value (true).

Examples:

```
int main(void)
{
    #if 1
        printf("Yabba Dabba Do!\n");
    #else
        printf("Zip-Bang!\n");
    #endif
    return 0;
}
```

Only "Yabba Dabba Do!" is printed.

```
int main(void)
{
    #if 1
        printf("Checkpoint1\n");
    #elif 1
        printf("Checkpoint2\n");
    #endif
    return 0;
}
```

Only "Checkpoint1" is printed. Note that if the first line is `#if 0`, then only "Checkpoint2" would be printed.

```
#if OS==1
```

```

    printf("Version 1.0");
#elif OS==2
    printf("Version 2.0");
#else
    printf("Version unknown");
#endif

```

Prints according to the setting of OS which is defined with a `#define`.

1.7.2 `#define`, `#undef`, `#ifdef`, `#ifndef`

The preprocessing directives `#define` and `#undef` allow the definition of identifiers which hold a certain value. These identifiers can simply be constants or a macro function. The directives `#ifdef` and `#ifndef` allow conditional compiling of certain lines of code based on whether or not an identifier has been defined.

Syntax:

```
#define identifier replacement-code
```

```
#undef identifier
```

```
#ifdef identifier
#else OR #elif
#endif
```

```
#ifndef identifier
#else OR #elif
#endif
```

`#ifdef identifier` is the same as `#if defined(identifier)`.

`#ifndef identifier` is the same as `#if !defined(identifier)`.

An identifier defined with `#define` is available anywhere in the source code until a `#undef` is reached.

A function macro can be defined with `#define` in the following manner:

```
#define identifier (parameter-list) (replacement-text)
```

The values in the *parameter-list* are replaced in the *replacement-text*.

Examples:

```
#define PI 3.141
printf("%f",PI);
```

```
#define DEBUG
#ifdef DEBUG
    printf("This is a debug message.");
#endif
```

```
#define QUICK(x) printf("%s\n",x);
QUICK("Hi!")
```

```
#define ADD(x, y) x + y
z=3 * ADD(5,6)
```

This evaluates to 21 due to the fact that multiplication takes precedence over addition.

```
#define ADD(x,y) (x + y)
z=3 * ADD(5,6)
```

This evaluates to 33 due to the fact that the summation is encapsulated in parenthesis which takes precedence over multiplication.

1.7.3 #include

The `#include` directive allows external header files to be processed by the compiler.

Syntax:

```
#include <header-file>

or

#include "source-file"
```

When enclosing the file with `<` and `>`, then the implementation searches the known header directories for the file (which is implementation-defined) and processes it. When enclosed with double quotation marks, then the entire contents of the source-file is replaced at this point. The searching manner for the file is implementation-specific.

Examples:

```
#include <stdio.h>
#include "my_header.h"
```

1.7.4 #line

The `#line` directive allows the current line number and the apparent name of the current sourcecode filename to be changed.

Syntax:

```
#line line-number filename
```

Note that if the filename is not given, then it stays the same. The line number on the current line is one greater than the number of new-line characters (so the first line number is 1).

Examples:

```
#line 50 user.c

#line 23
```

1.7.5 #error

The `#error` directive will cause the compiler to halt compiling and return with the specified error message.

Syntax:

```
#error message
```

Examples:

```
#ifndef VERSION
#error Version number not specified.
#endif
```

1.7.6 #pragma

This `#pragma` directive allows a directive to be defined. Its effects are implementation-defined. If the pragma is not supported, then it is ignored.

Syntax:

```
#pragma directive
```

1.7.7 Predefined Macros

The following macros are already defined by the compiler and cannot be changed.

- `__LINE__` A decimal constant representing the current line number.
- `__FILE__` A string representing the current name of the source code file.
- `__DATE__` A string representing the current date when compiling began for the current source file. It is in the format "mmm dd yyyy", the same as what is generated by the `asctime` function.
- `__TIME__` A string literal representing the current time when compiling began for the current source file. It is in the format "hh:mm:ss", the same as what is generated by the `asctime` function.
- `__STDC__` The decimal constant 1. Used to indicate if this is a standard C compiler.